

Kahlen Asha G. Damasco

```
#Variable,   Simple Assignment Operator,   Value
Intro       =       "My name is Asha and these are some quotes from favorite authors that inspire me"

#Variable,   Leftward Assignment Operator,   Value
Sylvia      <-      "Let me live, love, and say it well in good sentences."

#           Value ,                               Rightward Assignment Operator,   Variable
"You will never be happy if you continue to search for what happiness consists of."  ->      Albert

print(Intro)
print(Sylvia)
print(Albert)

... [1] "My name is Asha and these are some quotes from favorite authors that inspire me"
    [1] "Let me live, love, and say it well in good sentences."
    [1] "You will never be happy if you continue to search for what happiness consists of."
```

I was trying to find the easiest way to approach how to analyze and understand the syntax of R, and I found that focusing more on the comparison of it to Python is most effective since both are the two most popular programming languages for data science, both languages are well suited for any data science tasks and have been creating debate and discourse online. During my research I noticed that variables could go either in the front or the back, depending on what assignment operator you choose whether it be simple, leftward, or Rightward, but this is quite different from python's expressions and statements which must strictly follow a variable operator value structure. Operators are in-between and are usually similar to what you'd expect, like +, -, ==, and != for math and comparisons, so that part felt pretty familiar right away. What I learned is that R just loves giving you options like how you can write `x <- 5`, flip it as `5 -> x`, or go the Python route with `x = 5`, and they all mean the same thing, which felt kind of chaotic at first until I realized it says a lot about who built each language. R by statisticians who wanted something quick and flexible, Python built to be clean and predictable, locking you into one direction with no exceptions. R, meanwhile, kind of hands you the paintbrush and says do whatever works for you, which took some getting used to. Once that clicked, the syntax stopped looking like a random mess and started making sense as its own kind of logic. It also changed how I read code, since with R I can't just skim left to right, I have to look at the whole line first. In the end comparing the two side by side taught me neither language is actually better, but more that they're just built with what could be the same priorities but for different types of programmers and programming styles, and once I stopped expecting R to act like Python, everything got easier to understand.

References

Canales Luna, J. (2022, December 28). Python vs. R for data science: Which should you learn? DataCamp.

<https://www.datacamp.com/blog/python-vs-r-for-data-science-whats-the-difference>

Elsinghorst, S. (2017, January 22). R vs Python – A one-on-one comparison. Shirin's playgRound. https://shiring.github.io/r_vs_python/2017/01/22/R_vs_Py_post

GeeksforGeeks. (2026, February 20). Basic syntax in R programming.

<https://www.geeksforgeeks.org/r-language/basic-syntax-in-r-programming/>

GeeksforGeeks. (2025, July 23). Python syntax.

<https://www.geeksforgeeks.org/python/python-syntax/>

Google Colab Link

https://colab.research.google.com/drive/1R_xoG_Q_wgYjZP6wpTJUrgV6wReVxT5W?usp=sharing